



हरियाणा केंद्रीय विश्वविद्यालय
CENTRAL UNIVERSITY OF HARYANA
NAAC ACCREDITED 'A' GRADE UNIVERSITY

UNIT-1

MATLAB programming based optimization techniques implementation in power systems

Dr. Manish Kumar
Assistant Professor
Department of Electrical Engineering
School of Engineering and Technology
Central University of Haryana

Introduction to Optimization

- **Introduction**
 - Optimization is the act of obtaining the best result under given circumstances. Optimization can be defined as the process of finding the conditions that give the maximum or minimum value of a function.
- **Historical Development**
 - The existence of optimization methods can be traced to the days of Newton, Lagrange, and Cauchy.
 - The development of differential calculus methods of optimization was possible because of the contributions of Newton and Leibnitz.
 - The modern optimization methods, the genetic algorithms were originally proposed by John Holland in 1975.

Introduction to Optimization

Table 1.1 Methods of Operations Research

Mathematical programming or optimization techniques	Stochastic process techniques	Statistical methods
Calculus methods	Statistical decision theory	Regression analysis
Calculus of variations	Markov processes	Cluster analysis, pattern recognition
Nonlinear programming	Queueing theory	Design of experiments
Geometric programming	Renewal theory	Discriminate analysis (factor analysis)
Quadratic programming	Simulation methods	
Linear programming	Reliability theory	
Dynamic programming		
Integer programming		
Stochastic programming		
Separable programming		
Multiobjective programming		
Network methods: CPM and PERT		
Game theory		
<i>Modern or nontraditional optimization techniques</i>		
Genetic algorithms		
Simulated annealing		
Ant colony optimization		
Particle swarm optimization		
Neural networks		
Fuzzy optimization		

Statement Of An Optimization Problem

□ An optimization or a mathematical programming problem can be stated as follows

- Find $X = \{x_1, x_2, \dots, x_n\}^T$, which optimizes $f(X)$
- subject to the constraints

$$g_j(X) \leq 0, \quad j = 1, 2, \dots, m$$

$$l_k(X) = 0, \quad k = 1, 2, \dots, p$$

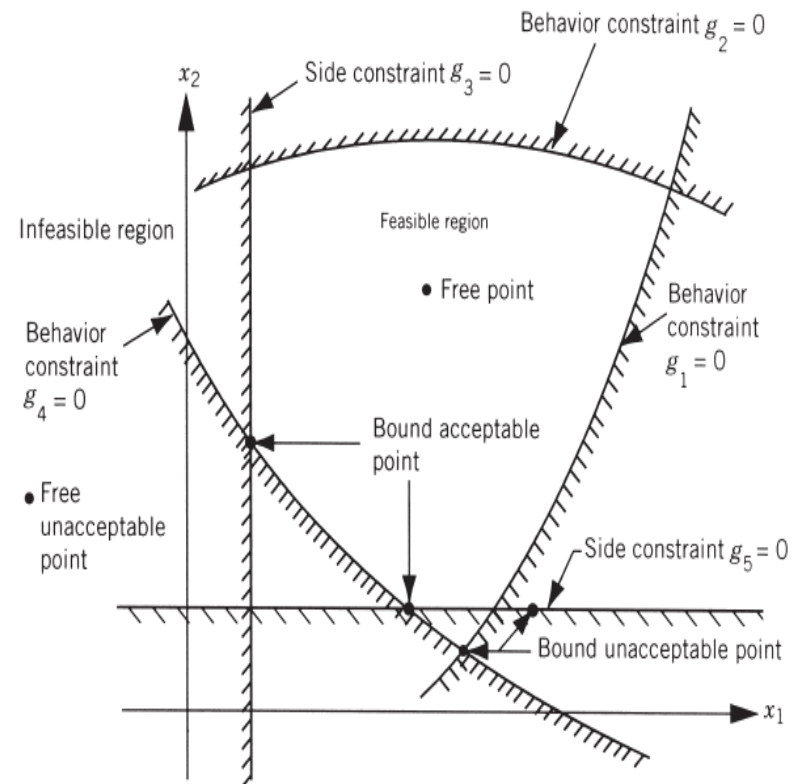
where X is an n -dimensional vector called the design vector, $f(X)$ is termed the objective function, and $g_j(X)$ and $l_k(X)$ are known as inequality and equality constraints, respectively. The number of variables n and the number of constraints m and/or p need not be related in any way.

□ Some optimization problems do not involve any constraints and can be stated as

- Find $X = \{x_1, x_2, \dots, x_n\}^T$, which optimizes $f(X)$
- Such problems are called unconstrained optimization problems.

Statement Of An Optimization Problem

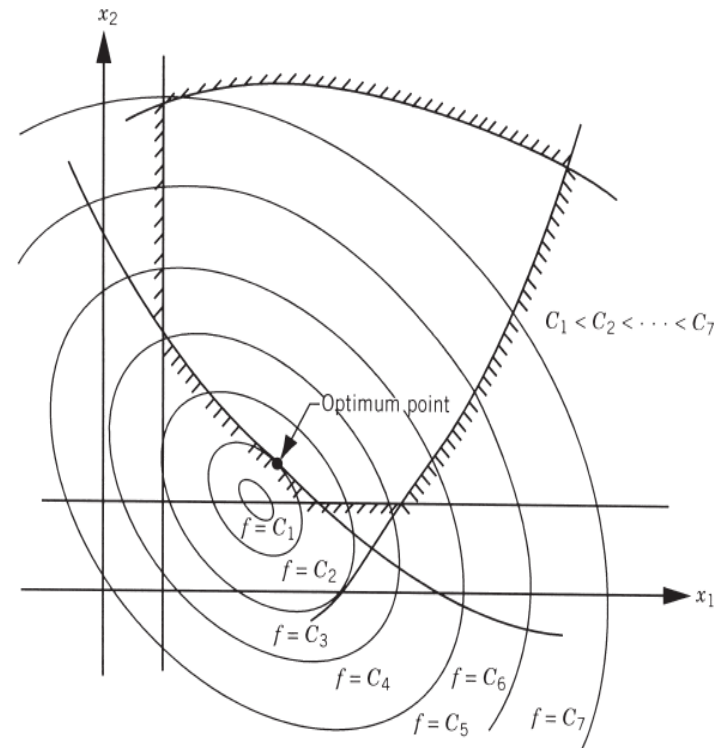
- Design Vector
 - Any engineering system or component is defined by a set of quantities some of which are viewed as variables during the design process.
- Design Constraints
 - The restrictions that must be satisfied to produce an acceptable design are collectively called design constraints.
 - Constraints that represent limitations on the behaviour or performance of the system are termed behaviour or functional constraints.
- Constraint Surface
 - Depending on whether a particular design point belongs to the acceptable or unacceptable region, it can be identified as one of the following four types:
 1. Free and acceptable point
 2. Free and unacceptable point
 3. Bound and acceptable point
 4. Bound and unacceptable point



Constraint surfaces in a hypothetical two-dimensional design space.

Statement Of An Optimization Problem

- **Objective Function**
 - The criterion with respect to which the design is optimized, when expressed as a function of the design variables, is known as the criterion or merit or objective function.
- **Multi-Objective**
 - An optimization problem involving multiple objective functions is known as a multi-objective programming problem.
 - Thus if $f_1(X)$ and $f_2(X)$ denote two objective functions, construct a new (overall) objective function for optimization as $f(X) = \alpha_1 f_1(X) + \alpha_2 f_2(X)$
- **Objective Function Surfaces**
 - The locus of all points satisfying $f(X) = C = \text{constant}$ forms a hyper surface in the design space, and each value of C corresponds to a different member of a family of surfaces. These surfaces, called objective function surfaces.



Contours of the objective function.

Classification Of Optimization Problems

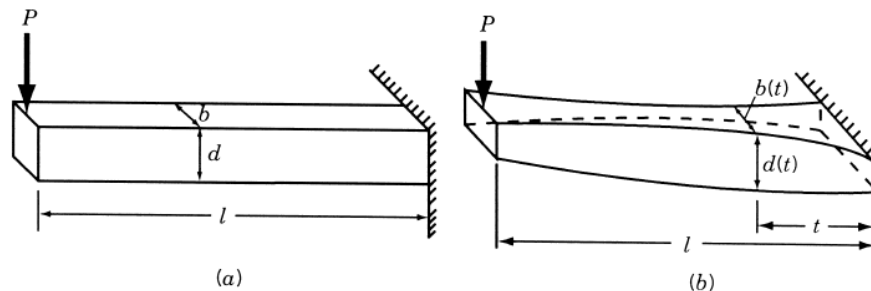
- Classification Based on the Existence of Constraints
 - Any optimization problem can be classified depending on whether constraints exist in the problem.
 1. **Constrained**
 2. **Unconstrained**

- Classification Based on the Nature of the Design Variables

1. Parameter or static optimization problems

The problem is to find values to a set of design parameters that make some prescribed function of these parameters minimum subject to certain constraints.

- For example, the problem of minimum-weight design of a prismatic beam shown in Fig. subject to a limitation on the maximum deflection can be stated as follows:
- Find $X = \{b \ d\}^T$ which minimizes, $f(X) = \rho l b d$
- Subject to the constraints: $\delta_{tip}(X) \leq \delta_{max}$, $b \geq 0$, $d \geq 0$.
- where ρ is the density and δ_{tip} is the tip deflection of the beam.



Cantilever beam under concentrated load.

Classification Of Optimization Problems

2. Trajectory or dynamic optimization problem

The objective is to find a set of design parameters, which are all continuous functions of some other parameter, that optimizes an objective function subject to a set of constraints. If the cross-sectional dimensions of the rectangular beam are allowed to vary along its length as shown in Fig.

- Find $\mathbf{X}(t) = \{b(t) \ d(t)\}^T$, which minimizes, $f[\mathbf{X}(t)] = \rho \int_0^l b(t) \ d(t) \ dt$
 - Subject to the constraints: $\delta_{tip}[\mathbf{X}(t)] \leq \delta_{max}, \quad 0 \leq t \leq l$
 $b(t) \geq 0, \quad 0 \leq t \leq l$
 $d(t) \geq 0, \quad 0 \leq t \leq l$
 - Here the design variables are functions of the length parameter t .
- Classification Based on the Nature of the Equations Involved
 1. Linear Programming Problem
 2. Nonlinear Programming Problem
 3. Geometric Programming Problem
 4. Quadratic Programming Problem

Convex and Concave Functions

Convex Function: A function $f(\mathbf{X})$ is said to be convex if for any pair of points $\mathbf{X}_1 = \{x_1^{(1)} \ x_2^{(1)} \ \dots \ x_n^{(1)}\}^T$ and $\mathbf{X}_2 = \{x_1^{(2)} \ x_2^{(2)} \ \dots \ x_n^{(2)}\}^T$ and all $\lambda, 0 \leq \lambda \leq 1$,

$$f[\lambda \mathbf{X}_2 + (1 - \lambda) \mathbf{X}_1] \leq \lambda f(\mathbf{X}_2) + (1 - \lambda) f(\mathbf{X}_1)$$

That is, if the segment joining the two points lies entirely above or on the graph of $f(\mathbf{X})$. Figures A.1a and A.2a illustrate a convex function in one and two dimensions, respectively. It can be seen that a convex function is always bending upward and hence it is apparent that the local minimum of a convex function is also a global minimum.

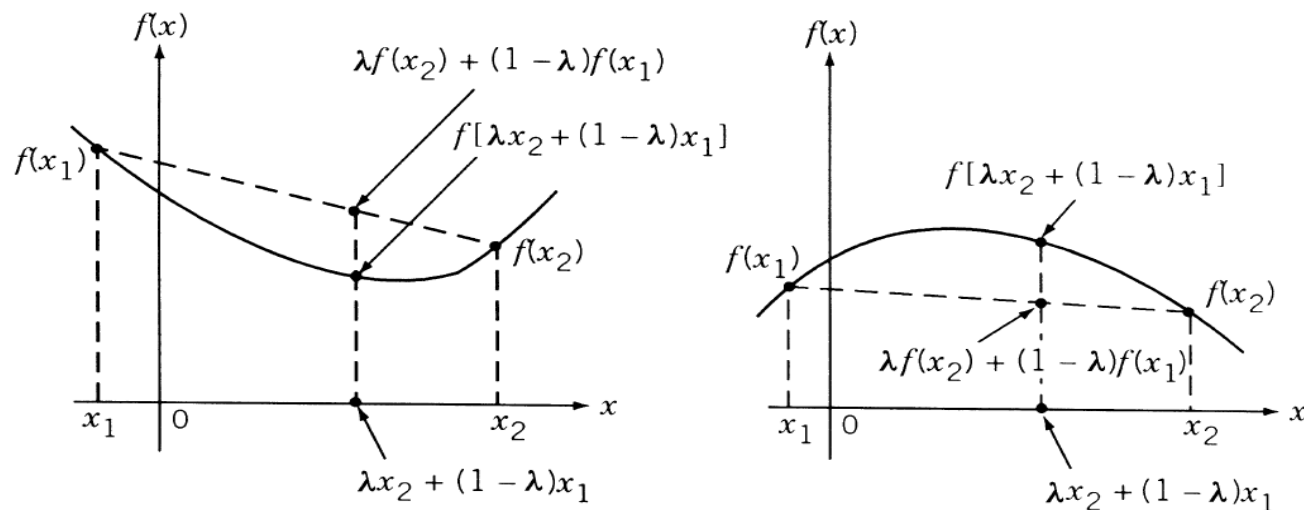


Figure A.1 Functions of one variable: (a) convex function in one variable; (b) concave function in one variable.

Convex and Concave Functions

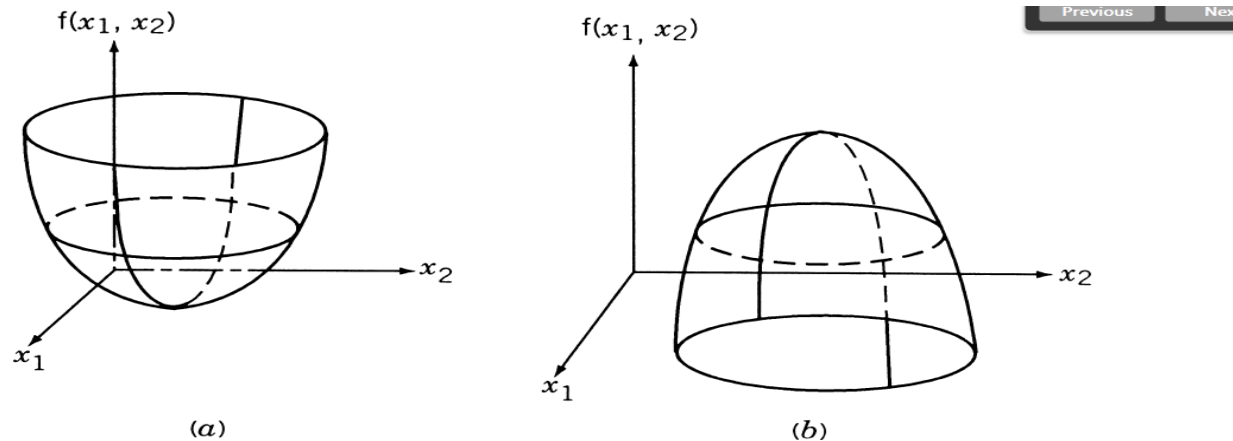


Figure A.2 Functions of two variables: (a) convex function in two variables; (b) concave function in two variables.

Concave Function: A function $f(X)$ is called a concave function if for any two points X_1 and X_2 , and for all $0 \leq \lambda \leq 1$,

$$f[\lambda X_2 + (1 - \lambda)X_1] \geq \lambda f(X_2) + (1 - \lambda)f(X_1)$$

That is, if the line segment joining the two points lies entirely below or on the graph of $f(X)$. Figures A.1b and A.2b give a concave function in one and two dimensions, respectively. It can be seen that a concave function bends downward and hence the local maximum will also be its global maximum.

Classical Optimization Techniques

- Introduction
 - The classical methods of optimization are useful in finding the optimum solution of continuous and differentiable functions. These methods are analytical and make use of the techniques of differential calculus in locating the optimum points.
- Single-variable Optimization
- Multivariable Optimization With No Constraints
- Multivariable Optimization With Equality Constraints
 - Solution by Direct Substitution
 - Solution by the Method of Constrained Variation
 - Solution by the Method of Lagrange Multipliers
- Multivariable Optimization With Inequality Constraints
 - Kuhn–Tucker Conditions
 - Constraint Qualification
- Convex Programming Problem

Operating Cost Of A Thermal Plant

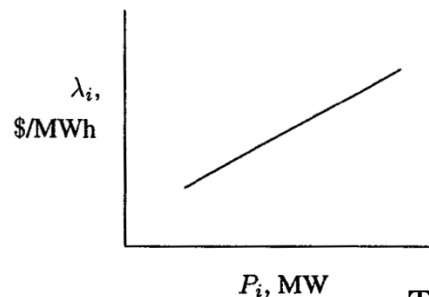
- The factors influencing power generation at minimum cost are operating efficiencies of generators, fuel cost, and transmission losses.
- The input to the thermal plant is generally measured in Btu/h, and the output is measured in MW.
- In all practical cases, the fuel cost of generator i can be represented as a quadratic function of real power generation:

$$C_i = \alpha_i + \beta_i P_i + \gamma_i P_i^2$$

- An important characteristic is obtained by plotting the derivative of the fuel-cost curve versus the real power. This is known as the incremental fuel-cost curve.

$$\frac{dC_i}{dP_i} = 2\gamma_i P_i + \beta_i$$

- The incremental fuel-cost curve is a measure of how costly it will be to produce the next increment of power.



Typical incremental fuel-cost curve.

Operating Cost Of A Thermal Plant

- The objective function (i.e., total production cost) as defined by the equation:

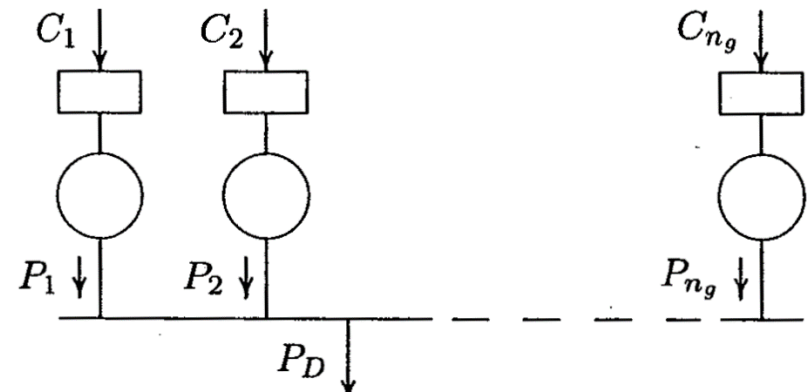
$$C_t = \sum_{i=1}^{n_g} C_i$$

$$= \sum_{i=1}^n \alpha_i + \beta_i P_i + \gamma_i P_i^2$$

- Subject to the constraint:

$$\sum_{i=1}^{n_g} P_i = P_D$$

$$P_i = \frac{\lambda - \beta_i}{2\gamma_i}$$



Economic Dispatch Neglecting Losses And Including Generator Limits

- The problem is to find the real power generation for each plant such that the objective function (i.e., total production cost) is minimum, subject to the constraint and the inequality constraints given by:

$$P_{i(min)} \leq P_i \leq P_{i(max)} \quad i = 1, \dots, n_g$$

- Where $P_{i(min)}$ and $P_{i(max)}$ are the minimum and maximum generating limits respectively for plant i .
- The Kuhn-Tucker conditions complement the Lagrangian conditions to include the inequality constraints as additional terms. The necessary conditions for the optimal dispatch with losses neglected becomes

$$\begin{aligned} \frac{dC_i}{dP_i} &= \lambda & \text{for } P_{i(min)} < P_i < P_{i(max)} \\ \frac{dC_i}{dP_i} &\leq \lambda & \text{for } P_i = P_{i(max)} \\ \frac{dC_i}{dP_i} &\geq \lambda & \text{for } P_i = P_{i(min)} \end{aligned}$$

Use Of The Gradient Method

$$f(\lambda) = P_D$$

- Expanding the left-hand side of the above equation in Taylor's series about an operating point $\lambda^{(k)}$ and neglecting the higher-order terms.

$$f(\lambda)^{(k)} + \left(\frac{df(\lambda)}{d\lambda} \right)^{(k)} \Delta\lambda^{(k)} = P_D$$

$$\begin{aligned} \Delta\lambda^{(k)} &= \frac{\Delta P^{(k)}}{\left(\frac{df(\lambda)}{d\lambda} \right)^{(k)}} \\ &= \frac{\Delta P^{(k)}}{\sum \left(\frac{dP_i}{d\lambda} \right)^{(k)}} \end{aligned}$$

$$\Delta\lambda^{(k)} = \frac{\Delta P^{(k)}}{\sum \frac{1}{2\gamma_i}}$$

$$\lambda^{(k+1)} = \lambda^{(k)} + \Delta\lambda^{(k)}$$

$$\Delta P^{(k)} = P_D - \sum_{i=1}^{n_g} P_i^{(k)}$$

The process is continued until $\Delta P^{(k)}$ is less than a specified accuracy

Experiment

No. : - 1

- Objective:- Computation of scheduling of thermal power plants without network losses
- Problem 1- Find the optimal dispatch and the total fuel cost of generation in \$/hr, for the thermal plants whose input data are given below:

$$C_1 = 500 + 5.3P_1 + 0.004P_1^2$$

$$C_2 = 400 + 5.5P_2 + 0.006P_2^2$$

$$C_3 = 200 + 5.8P_3 + 0.009P_3^2$$

$$200 \leq P_1 \leq 450$$

$$150 \leq P_2 \leq 350$$

$$100 \leq P_3 \leq 2250$$

Assume system total load demand is 975 MW.

Solution:-

Solution:-

$\alpha = [500; 400; 200];$

$\beta = [5.3; 5.5; 5.8];$

$\gamma = [0.004; 0.006; 0.009];$

$P_{\max} = [450 \ 350 \ 225];$

$P_{\min} = [200 \ 150 \ 100];$

$NoGen = \text{size}(\gamma, 1);$

$PD = 975;$

$\Delta p = 10;$

$\lambda = 9;$

$\text{disp}(' \text{Lambda } P1 \ P2 \ P3 \ DP \ grad \ \Delta \lambda');$

$Iter = 0;$

```

while abs(Delp) .>= 0.0001
    iter = iter+1;
    P = (lambda - beta) ./ (2*gamma);
    for n = 1: NoGen
        if (P(n) > Pmax(n))
            P(n) = Pmax(n);
        elseif (P(n) < Pmin(n))
            P(n) = Pmin(n);
        else
            P(n) = P(n);
        end
    end
    Delp = abs(PD - sum(P));
    J = sum(ones(length(gamma),1)./(2*gamma));
    Delambda = Delp/J;
    disp([lambda, P(1), P(2), P(3), Delp, J, Delambda]);
    if (PD > sum(p))
        Lambda = lambda + Delambda;
    elseif (PD < sum(P))
        Lambda = lambda - Delambda;
    end
end
Total cost = sum(alpha + beta.*P + gamma.*P.^2)

```

Economic Dispatch Including Losses

- The simplest quadratic form is

$$P_L = \sum_{i=1}^{n_g} \sum_{j=1}^{n_g} P_i B_{ij} P_j$$

- A more general formula containing a linear term and a constant term, referred to as Kron's loss formula, is

$$P_L = \sum_{i=1}^{n_g} \sum_{j=1}^{n_g} P_i B_{ij} P_j + \sum_{i=1}^{n_g} B_{0i} P_i + B_{00}$$

- The coefficients B_{ij} are called loss coefficients or B -coefficients.
- The function of plant output

$$\begin{aligned} C_t &= \sum_{i=1}^{n_g} C_i \\ &= \sum_{i=1}^n \alpha_i + \beta_i P_i + \gamma_i P_i^2 \end{aligned}$$

- Subject to the constraint that generation should equal total demands plus losses

$$\sum_{i=1}^{n_g} P_i = P_D + P_L$$

Economic Dispatch Including Losses

- Using the Lagrange multiplier and adding additional terms to include the inequality constraints, we obtain

$$\mathcal{L} = C_t + \lambda(P_D + P_L - \sum_{i=1}^{n_g} P_i) + \sum_{i=1}^{n_g} \mu_{i(max)}(P_i - P_{i(max)}) + \sum_{i=1}^{n_g} \mu_{i(min)}(P_i - P_{i(min)})$$

- The constraints should be understood to mean the $\mu_{i(max)} = 0$ when $P_i < P_{i(max)}$ and that $\mu_{i(min)} = 0$ when $P_i > P_{i(min)}$.
- The minimum of this unconstrained function is found at the point where the partials of the function to its variables are zero.

$$+ \frac{\partial \mathcal{L}}{\partial P_i} = 0$$

$$+ \frac{\partial \mathcal{L}}{\partial \lambda} = 0$$

$$+ \frac{\partial \mathcal{L}}{\partial \mu_{i(max)}} = P_i - P_{i(max)} = 0$$

$$+ \frac{\partial \mathcal{L}}{\partial \mu_{i(min)}} = P_i - P_{i(min)} = 0$$

$$+ \frac{\partial C_t}{\partial P_i} + \lambda(0 + \frac{\partial P_L}{\partial P_i} - 1) = 0$$

$$+ C_t = C_1 + C_2 + \dots + C_{n_g}$$

$$+ \frac{\partial C_t}{\partial P_i} = \frac{dC_i}{dP_i}$$

Economic Dispatch Including Losses

- The condition for optimum dispatch is:

$$+ \frac{dC_i}{dP_i} + \lambda \frac{\partial P_L}{\partial P_i} = \lambda \quad i = 1, \dots, n_g$$

- The term $\frac{\partial P_L}{\partial P_i}$ is known as the incremental transmission loss.

$$+ \sum_{i=1}^{n_g} P_i = P_D + P_L$$

$$+ \left(\frac{1}{1 - \frac{\partial P_L}{\partial P_i}} \right) \frac{dC_i}{dP_i} = \lambda \quad i = 1, \dots, n_g$$

$$+ L_i \frac{dC_i}{dP_i} = \lambda \quad i = 1, \dots, n_g$$

- where L_i is known as the penalty factor of plant i and is given by $L_i = \frac{1}{1 - \frac{\partial P_L}{\partial P_i}}$

$$+ \frac{\partial P_L}{\partial P_i} = 2 \sum_{j=1}^{n_g} B_{ij} P_j + B_{0i}$$

$$+ \beta_i + 2\gamma_i P_i + 2\lambda \sum_{j=1}^{n_g} B_{ij} P_j + B_{0i} \lambda = \lambda$$

$$+ \left(\frac{\gamma_i}{\lambda} + B_{ii} \right) P_i + \sum_{\substack{j=1 \\ j \neq i}}^{n_g} B_{ij} P_j = \frac{1}{2} \left(1 - B_{0i} - \frac{\beta_i}{\lambda} \right)$$

$$+ \begin{bmatrix} \frac{\gamma_1}{\lambda} + B_{11} & B_{12} & \cdots & B_{1n_g} \\ B_{21} & \frac{\gamma_2}{\lambda} + B_{22} & \cdots & B_{2n_g} \\ \vdots & \vdots & \ddots & \vdots \\ B_{n_g 1} & B_{n_g 2} & \cdots & \frac{\gamma_{n_g}}{\lambda} + B_{n_g n_g} \end{bmatrix} \begin{bmatrix} P_1 \\ P_2 \\ \vdots \\ P_{n_g} \end{bmatrix} = \frac{1}{2} \begin{bmatrix} 1 - B_{01} - \frac{\beta_1}{\lambda} \\ 1 - B_{02} - \frac{\beta_2}{\lambda} \\ \vdots \\ 1 - B_{0n_g} - \frac{\beta_{n_g}}{\lambda} \end{bmatrix}$$

Economic Dispatch Including Losses^[2]

- So, $EP=D$
- P_i at the k^{th} iteration is expressed as:

$$★ \quad P_i^{(k)} = \frac{\lambda^{(k)}(1 - B_{0i}) - \beta_i - 2\lambda^{(k)} \sum_{j \neq i} B_{ij} P_j^{(k)}}{2(\gamma_i + \lambda^{(k)} B_{ii})}$$

$$★ \quad \sum_{i=1}^{n_g} \frac{\lambda^{(k)}(1 - B_{0i}) - \beta_i - 2\lambda^{(k)} \sum_{j \neq i} B_{ij} P_j^{(k)}}{2(\gamma_i + \lambda^{(k)} B_{ii})} = P_D + P_L^{(k)}$$

$$★ \quad f(\lambda)^{(k)} = P_D + P_L^{(k)}$$

$$★ \quad f(\lambda)^{(k)} + \left(\frac{df(\lambda)}{d\lambda} \right)^{(k)} \Delta\lambda^{(k)} = P_D + P_L^{(k)}$$

$$\begin{aligned} ★ \quad \Delta\lambda^{(k)} &= \frac{\Delta P^{(k)}}{\left(\frac{df(\lambda)}{d\lambda} \right)^{(k)}} \\ &= \frac{\Delta P^{(k)}}{\sum \left(\frac{dP_i}{d\lambda} \right)^{(k)}} \end{aligned}$$

$$★ \quad \sum_{i=1}^{n_g} \left(\frac{\partial P_i}{\partial \lambda} \right)^{(k)} = \sum_{i=1}^{n_g} \frac{\gamma_i(1 - B_{0i}) + B_{ii}\beta_i - 2\gamma_i \sum_{j \neq i} B_{ij} P_j^{(k)}}{2(\gamma_i + \lambda^{(k)} B_{ii})^2}$$

$$★ \quad \lambda^{(k+1)} = \lambda^{(k)} + \Delta\lambda^{(k)}$$

$$★ \quad \Delta P^{(k)} = P_D + P_L^{(k)} - \sum_{i=1}^{n_g} P_i^{(k)}$$

Economic Dispatch Including Losses

- An approximate loss formula expressed by

- ✦ $P_L = \sum_{i=1}^{n_g} B_{ii} P_i^2$

- ✦ $P_i^{(k)} = \frac{\lambda^{(k)} - \beta_i}{2(\gamma_i + \lambda^{(k)} B_{ii})}$

- ✦ $\sum_{i=1}^{n_g} \left(\frac{\partial P_i}{\partial \lambda} \right)^{(k)} = \sum_{i=1}^{n_g} \frac{\gamma_i + B_{ii} \beta_i}{2(\gamma_i + \lambda^{(k)} B_{ii})^2}$

Experiment

- No. : - 2
- Objective: - Economic load dispatch considering network losses.
- Problem:- The fuel costs of three thermal power generators of a Power system are:

$$C_1 = 200 + 7.0 P_1 + 0.008 P_{12} \text{ $/hr};$$

$$C_2 = 180 + 6.3 P_2 + 0.008 P_{22} \text{ $/hr};$$

$$C_3 = 140 + 6.8 P_3 + 0.007 P_{32} \text{ $/hr};$$

Where P_1 , P_2 , P_3 are in MW. Plant outputs are subject to following limits;

$$10 \text{ MW} \leq P_1 \leq 85 \text{ MW}$$

$$10 \text{ MW} \leq P_2 \leq 80 \text{ MW}$$

$$10 \text{ MW} \leq P_3 \leq 70 \text{ MW}$$

Real Power loss has been considered in this case, where values of loss coefficients are mentioned below;

$$B = \begin{bmatrix} 0.0218 & 0 & 0 \\ 0 & 0.0228 & 0 \\ 0 & 0 & 0.0179 \end{bmatrix};$$

Loss Coefficients are specified in p.u on a 100 MVA base.

Determine the optimal dispatch of generation when the total system load is 150 MW.

Solution:

```
clear all;
cost = [200 7.0 0.008;
        180 6.3 0.009;
        140 6.8 0.007];
mwlimits = [10 85;
            10 80;
            10 70];
pdt = 150;
B = [0.0218 0 0;
     0 0.0228 0;
     0 0 0.0179];
Bo = [0 0 0];
Boo = [0];
basemva = 100;
ngg = length(cost(:,1));
```

% Economic Power Output Calculation

```
Bu = B/basemva;
Boou = basemva*Boo;
alpha = cost(:,1);
beta = cost(:, 2);
gama = cost(:,3);
Pmin = mwlimits(:,1);
Pmax = mwlimits(:,2);
```

```

iterp = 0; %Iteration Counter
Delp = 10; %Error in Delp is set to a high value
E = Bu;
lambda=10;
while (abs(Delp) >= 0.00001) % Test for convergence
    iterp = iterp + 1; % No, of iterations

    for k = 1: ngg
        E(k,k) = gama(k)/ lambda + Bu(k,k);
        Dx(k) = 1/2*(1-Bo(k)-beta(k)/lambda);
        for m=1:ngg
            if(m~=k)
                E(k,m)=0;
            end
        end
    end
    PP=inv(E)*Dx';

    Pgg=PP';

    for k = 1:ngg
        if((Pgg(k)>Pmax(k)))
            Pgg(k) = Pmax(k);
        elseif((Pgg(k)< Pmin(k)))
            Pgg(k) = Pmin(k);
        end
    end
end

```

```
PL=Pgg*Bu*Pgg' + Bo*Pgg' + Boou;
```

```
Delp = pdt + PL - sum(Pgg);
```

```
for k= 1:ngg
```

```
    BP = 0 ;
```

```
    for m= 1:ngg
```

```
        if (m~=k)
```

```
            BP = BP + Bu (k,m)*Pgg(m);
```

```
        end
```

```
    end
```

```
    grad(k) = (gama(k)*(1-Bo(k))+ Bu(k,k)*beta(k)- (2* gama(k)*BP))/(2*(gama(k) + lambda*Bu(k,k))^2);
```

```
end
```

```
sumgrad =sum(grad);
```

```
Delambda = Delp /sumgrad;
```

```
%Change in variable
```

```
lambda = lambda + Delambda;
```

```
% Successive solution
```

```
end
```

```
PLoss =Pgg*Bu*Pgg' + Bo*Pgg' + Boou;
```

```
Costv = cost(:,1) + cost(:,2).*Pgg' + cost(:,3).*(Pgg').^2;
```

```
Totalcost = sum(Costv);
```

Output:

```
Loss =1.6991 MW;
```

```
Cost =1592.65;
```

```
35.0907 MW
```

```
64.1317MW
```

```
52.4767 MW
```

```
lambda= 7.678935$ hr.
```

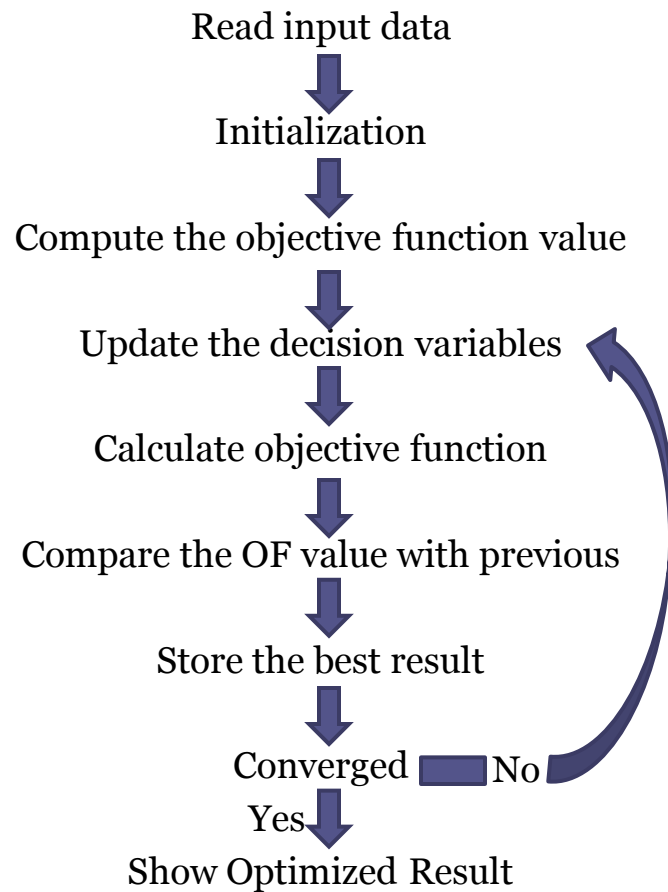
Soft Computing Techniques



Modern Methods of Optimization (Soft-Computing)

- Most of these methods are based on certain characteristics and behavior of biological, molecular, swarm of insects, and neurobiological systems.
- Such as:
 - Genetic algorithms
 - Particle swarm optimization
 - Simulated annealing
 - Ant colony optimization
 - Fuzzy optimization
 - Neural-network-based methods

Basic Steps for Optimization



Economic Load Dispatch (ELD)

- Power system Economic Load Dispatch (ELD) is the process of allocating generation among the available generating units such that the cost of operation is least, subject to load demand and other operational constraints.
- The basic Economic Load Dispatch considers
 - Power balance constraint
 - Generating capacity limits
- To provide the completeness for the economic dispatch formulation a practical Economic Dispatch considers
 - Ramp-rate limits
 - Prohibited operating zones
 - Valve-point effects
 - Multi-fuel options

Economic Load Dispatch (ELD)

- The fuel cost of each generator is generally expressed as a quadratic function of that generator output and is expressed in the form

$$F_i(P_i) = a_i + b_i P_i + c_i P_i^2 \quad (\$/hr)$$

- where, a_i, b_i, c_i are the quadratic cost coefficients of i^{th} generating unit.
 f_i is the fuel cost of i^{th} generator (P_i).
- The operating cost of all the N generators in the system is therefore,

$$f_1 = \sum_{i=1}^N F_i(P_i) \quad (\$/hr)$$

Generalized Constraints of ELD

- Real Power Balance constraint

- With Transmission Loss:-
$$\sum_{i=1}^N P_i - P_D - P_L = 0$$

- Without Transmission Loss:-
$$\sum_{i=1}^N P_i - P_D = 0$$

where, P_D is the power demand
 P_L is the real power losses

- The Transmission loss may be evaluated by Kron's Loss formula:

$$P_L = \sum_{i=1}^N \sum_{j=1}^N P_i B_{ij} P_j + \sum_{i=1}^N B_{0i} P_i + B_{00}$$

- Real Power Operating Limits

$$P_{i \min} \leq P_i \leq P_{i \max} \quad i = 1, 2, \dots, N$$

where,

$P_{i \min}$ and $P_{i \max}$ are the minimum and maximum capacity of i^{th} generator.

P_i is the output of the i^{th} unit at any instant of time.

Generalized Constraints of ELD

- Generator Ramp Rate Limits

- Ramp-rate limits is included in order to keep thermal gradients inside the turbine within safe limits and to avoid shortening of life of the generators.
- As generation increases: $P_i - P_{i0} \leq UR_i$
- As generation decreases: $P_{i0} - P_i \leq DR_i$

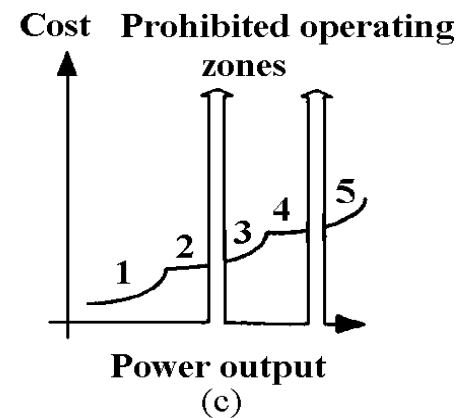
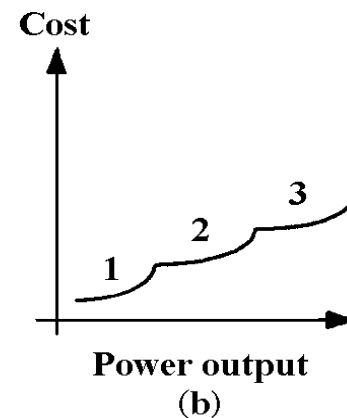
where, P_i (i^{th} generating unit) is the power output in the current interval.

P_{i0} is the power output in the previous interval.

UR_i , DR_i are the Up and Down Ramp-rate limits of i^{th} Generator.

- Generator Prohibited Operating Zones

- Mainly develop due to
 - faults in the machines, boilers, feed pumps and steam valve operation
 - vibration in the shaft bearing etc.



ELD Initialization in Matlab Programming

```
% *****
% initialization of population
% *****

I_NP=10;
I_D=8
FM_pop = zeros(I_NP,I_D);
FVr_minbound= [234 63 50 76 100 21 100 120];
FVr_maxbound=[2340 630 500 760 1000 210 1000 1200];
d=0;
izx = 1;
while ( izx ~= (I_NP + 1) )
    jvgk=1;
    while (jvgk ~= I_D)
        d(izx,jvgk) = (FVr_minbound(1,jvgk) + (FVr_maxbound(1,jvgk) - FVr_minbound(1,jvgk)) * rand);
        jvgk=jvgk+1;
    end
    x = sum((d(izx,:))')';
    y = PDx - x;

    if (y <= FVr_maxbound(1,jvgk) & y >= FVr_minbound(1,jvgk))
        d(izx,jvgk)=y;
        FM_pop(izx,:) = d(izx,:);
        izx = izx+1
    end
end
end
```

ELD Initialization in Matlab Programming

```
% ***** calculation of Cost Function
%*****

A3 = [0.005000000000000000    0.005500000000000000
0.006000000000000000    0.005000000000000000    0.005000000000000000
0.007000000000000000    0.007000000000000000    0.070000000000000000];
%'a' Coefficient Value

B3 = [1.8900000000000000    2.0000000000000000
3.5000000000000000    3.1500000000000000    3.0500000000000000
2.7500000000000000    3.4500000000000000    3.4500000000000000];
%'b' Coefficient Value

C3 = [150    115    40    122    125    120    70    70];
%'c' Coefficient Value

E3 = [300    200    200    150    150    150    200    150];
F3 = [0.0350000000000000    0.0420000000000000
0.0420000000000000    0.0630000000000000    0.0630000000000000
0.0630000000000000    0.0530000000000000    0.0630000000000000];
A4= repmat(A3,I_NP, 1);
B4= repmat(B3,I_NP, 1);
C4= repmat(C3,I_NP, 1);
E4= repmat(E3,I_NP, 1);
F4= repmat(F3,I_NP, 1);
```

```

% MinParValue1 = [50      50      20      20      25      40      45      35
25      25];          % Lower bound for Generator
Capacity
% MaxParValue1 = [455      450      130      130      470      460      465      300
160      180];          % Upper bound for Generator
Capacity
MinParValue1 = [234 63 50 76 100 21 100 120];
% Lower bound for Generator Capacity
MaxParValue1 = [2340 630 500 760 1000 210 1000 1200];
% Upper bound for Generator Capacity
i9 = 1;
while ( i9 ~= (I_NP + 1) )
    j9 = 1;
    while ( j9 ~= (I_D + 1) )
        G3(i9,j9) = sin((F4(i9,j9))*(MinParValue1(1,j9) -
FM_pop(i9,j9)));
        j9 = j9 + 1;
    end
    i9 = i9 + 1;
end
G4 = abs((E4).*(G3));
% ***** Cost Function
*****
Dejed = sum((A4.*(FM_pop.^2) + B4.*(FM_pop) + C4 )')' +
sum((G4)')';

```

Differential Evolution

- Differential Evolution
 - Differential Evolution is a simple population based Evolutionary Algorithm, capable of handling non-differentiable, non-linear, and multi-modal objective functions.
- Three basic operators
 - 1. Mutation, 2. Crossover, 3. Selection
- **Initialization**
 - The population is initialized randomly by generating individuals within the boundary constraints.

$$X_{ij}^0 = X_j^{\min} + rand \times (X_j^{\max} - X_j^{\min}) \quad \text{where } i = 1, 2, 3, \dots, N_p; j = 1, 2, 3, \dots, D;$$
 - Where, X_{ij}^0 is the initialized j^{th} decision variable of i^{th} population set; 'rand' function generates random values uniformly in the interval [0, 1]; N_p is the size of the population; D is the number of decision variables. $X_j^{\min}$ and $X_j^{\max}$ are the lower and upper bound of the j^{th} decision variable respectively.
- **Mutation (Differentiation)**
 - Mutation consists of two simultaneous operations:
 - (i) The choice of a 'direction' for mutation.
 - (ii) The calculation of a 'step length' in which this mutation performs.

Differential Evolution

- The mutation operator creates mutant vectors $X_i^{/k}$ by perturbing a randomly selected vector X_a^k with the difference of two other randomly selected vectors X_b^k and X_c^k at k^{th} iteration as per following equation.

$$X_i^{/k} = X_a^k + F \times (X_b^k - X_c^k); i=1,2,\dots,Np$$

- Where, $X_i^{/k}$ is the newly generated i^{th} population set after performing mutation operation at k^{th} iteration; X_a^k , X_b^k and X_c^k are randomly chosen vectors at k^{th} iteration $\in [i=1,2,3,\dots,Np]$ and $a \neq b \neq c \neq i$
- X_a^k , X_b^k and X_c^k are selected anew for each parent vector. $F \in [0, 2]$ is known as 'Scaling factor' used to control the amount of perturbation in the mutation process and improve convergence.

- Crossover (Recombination)**

- Crossover represents a typical case of a 'genes' exchange. The 'trial' one inherits genes with some probability.

$$X_{ij}^{//k} = \begin{cases} X_{ij}^{/k} & \text{if } randj < CR \text{ or } j = q \\ X_{ij}^k & \text{otherwise} \end{cases}$$

- Where, X_{ij}^k , $X_{ij}^{/k}$, $X_{ij}^{//k}$ are j^{th} individual of i^{th} parent vector, mutant vector, and trial vector at k^{th} iteration respectively. 'q' is a randomly chosen index $\in (j=1,2,\dots,D)$ that guarantees that the trial vector gets at least one parameter from the mutant vector even if $CR = 0$. $CR \in [0, 1]$ is the 'Crossover constant' that controls the diversity of the population and aids the algorithm to escape from local optima.

Differential Evolution

- **Selection**

- Selection procedure is used among the set of trial vector and the updated parent vector to choose the best.

$$X_i^{k+1} = \begin{cases} X_i^{//k} & \text{if } f(X_i^{//k}) \leq f(X_i^k) \\ X_i^k & \text{otherwise} \end{cases} ; i = 1, 2, \dots, Np$$

Algorithm 1:- The pseudo-code of the original DE algorithm with ‘Strategy1’

```

1:  Generate the initial population  $P$ 
2:  Evaluate the fitness for each individual in  $P$ 
3:  while the termination criterion is not satisfied
4:      for  $i = 1$  to  $N_p$ 
5:          Select 3 uniformly distributed integer random numbers
             $r_1, r_2$  and  $r_3$  in between 1 to  $N_p$ , such that  $r_1 \neq r_2 \neq r_3 \neq i$ 
6:           $j_{\text{rand}} = \text{randint}(1, D)$ 
7:          for  $j = 1$  to  $D$ 
8:              if  $\text{rand}_j(0, 1) > CR$  or  $j == j_{\text{rand}}$ 
9:                   $U_i(j) = X_{r1}(j) + F \times (X_{r2}(j) - X_{r3}(j))$ 
10:             else
11:                  $U_i(j) = X_i(j)$ 
12:             end
13:         end
14:     end
15:     for  $i = 1$  to  $N_p$ 
16:         Evaluate the offspring  $U_i$ 
17:         if  $U_i$  is better than  $P_i$ 
18:              $P_i = U_i$ 
19:         end
20:     end
21: end

```

Where D is the number of decision variables; N_p is the size of the parent population P ; $X_i(j)$ is the j^{th} variable of the solution X_i ; U_i is the offspring. ‘randint (1, D)’ is a uniformly distributed random integer number between 1 and D .

THANK YOU